

Hallucination Detection in Large Language Models with Metamorphic Relations

BORUI YANG, King's College London, United Kingdom

MD AFIF AL MAMUN, University of Calgary, Canada

JIE M. ZHANG, King's College London, United Kingdom

GIAS UDDIN, York University, Canada

Large Language Models (LLMs) are prone to hallucinations, e.g., factually incorrect information, in their responses. These hallucinations present challenges for LLM-based applications that demand high factual accuracy. Existing hallucination detection methods primarily depend on external resources, which can suffer from issues such as low availability, incomplete coverage, privacy concerns, high latency, low reliability, and poor scalability. There are also methods depending on output probabilities, which are often inaccessible for closed-source LLMs like GPT models. This paper presents MetaQA, a self-contained hallucination detection approach that leverages metamorphic relation and prompt mutation. Unlike existing methods, MetaQA operates without any external resources and is compatible with both open-source and closed-source LLMs. MetaQA is based on the hypothesis that if an LLM's response is a hallucination, the designed metamorphic relations will be violated. We compare MetaQA with the state-of-the-art zero-resource hallucination detection method, SelfCheckGPT, across multiple datasets, and on two open-source and two closed-source LLMs. Our results reveal that MetaQA outperforms SelfCheckGPT in terms of precision, recall, and f1 score. For the four LLMs we study, MetaQA outperforms SelfCheckGPT with a superiority margin ranging from 0.041 - 0.113 (for precision), 0.143 - 0.430 (for recall), and 0.154 - 0.368 (for F1-score). For instance, with Mistral-7B, MetaQA achieves an average F1-score of 0.435, compared to SelfCheckGPT's F1-score of 0.205, representing an improvement rate of 112.2%. MetaQA also demonstrates superiority across all different categories of questions.

CCS Concepts: • **Computing methodologies** → **Natural language processing**; • **Software and its engineering** → *Software testing and debugging*.

Additional Key Words and Phrases: Large Language Models, Hallucination Detection, Metamorphic Relations

ACM Reference Format:

Borui Yang, Md Afif Al Mamun, Jie M. Zhang, and Gias Uddin. 2025. Hallucination Detection in Large Language Models with Metamorphic Relations. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE020 (July 2025), 21 pages. <https://doi.org/10.1145/3715735>

1 Introduction

Large Language Models (LLMs) like GPT-4 [34] have revolutionized language processing by generating fluent and organized responses for various applications, such as drafting reports and summarization systems [23, 33, 52]. However, LLMs are prone to generating hallucinations—coherent but factually incorrect or irrelevant outputs, such as non-factual responses in question-answering context [16, 45]. This tendency poses significant challenges to the reliability of LLMs, undermining

Authors' Contact Information: Borui Yang, King's College London, London, United Kingdom, zbybr@bupt.edu.cn; Md Afif Al Mamun, University of Calgary, Calgary, Canada, afif.mamun@ucalgary.ca; Jie M. Zhang, King's College London, London, United Kingdom, jie.zhang@kcl.ac.uk; Gias Uddin, York University, Toronto, Canada, guddin@yorku.ca.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2994-970X/2025/7-ARTFSE020

<https://doi.org/10.1145/3715735>

the effectiveness of LLMs in applications requiring high factual accuracy. Fact-conflicting hallucinations, where LLMs produce content that contradicts established facts, are particularly concerning as they can mislead users lacking expertise on the topic, leading to significant confusion and eroding the trust essential for various LLM applications. Figure 1 illustrates a scenario where a legal question is posed to ChatGPT, which generates a response containing hallucinations. Without proper detection, such inaccuracies could have serious consequences for individuals who lack expertise in the legal field.

Many approaches have been introduced to detect or mitigate hallucination of LLMs that compare responses to factual information, often relying on databases or search engines [5, 6, 13–15, 17]. However, relying on external resources for hallucination detection often limits the scope to specific domains where a comprehensive database does not exist. Moreover, hallucinations are observed across a wide range of tasks that extend beyond simple fact verification [22]. Other works use token-level information such as token confidence and entropy [42, 47], which is often inaccessible in closed-source models. To tackle these issues, Manakul et al. proposed SelfcheckGPT [30], a self-contained approach to addressing hallucination. However, for the sample generation process in SelfCheckGPT, the LLM tends to produce samples that are identical to the original response, which significantly impacts its performance in hallucination detection.

This paper introduces MetaQA, a novel zero-resource technique that employs Metamorphic Relations (MRs) to detect hallucinations in LLM responses. MetaQA uses MRs to generate response mutations and verifies these mutations against expected outcomes to identify inconsistencies. Acting as a test oracle—similar to mechanisms in software testing, MetaQA ensures reliable factual accuracy checks of the LLM outputs without requiring additional agents. This method is advantageous as it relies solely on the LLM itself. MetaQA is applicable to both open-source and closed-source LLMs. MetaQA can be used without the need for intermediate processes or external tools.

We evaluate MetaQA using three datasets—TruthfulQA, HotpotQA, and FreshQA—the widely studied benchmarks in hallucination evaluation, across four LLMs: GPT-4, GPT-3.5, Llama3, and Mistral. Our results reveal that MetaQA consistently outperforms SelfCheckGPT in terms of precision, recall, and f1 score on all the four LLMs we study. In particular, for the four LLMs, MetaQA outperforms SelfCheckGPT with a superiority margin ranging from 0.154 to 0.368 in terms of F1-score. Our ablation studies also show that MetaQA has considerable stability across multiple runs, and has better performance with lower temperatures.

Our major contributions include:

- (1) To the best of our knowledge, we are the first to apply synonym and antonym-based metamorphic relations to detect hallucination LLMs responses.
- (2) We improve the TruthfulQA [28] benchmark by updating 238 questions with new correct answers. We share the improved benchmark, which we name *TruthfulQA-Enhanced*. This new benchmark can support more accurate hallucination detection research.
- (3) We conduct a large-scale evaluation of MetaQA on TruthfulQA-Enhanced, FreshQA, and HotpotQA datasets, showing superior performance over the baseline method SelfCheckGPT.

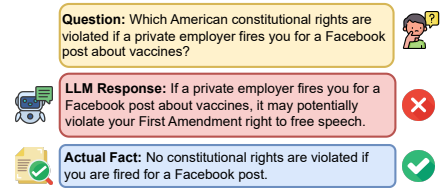


Fig. 1. An example of a hallucinated output generated by ChatGPT in the legal domain.

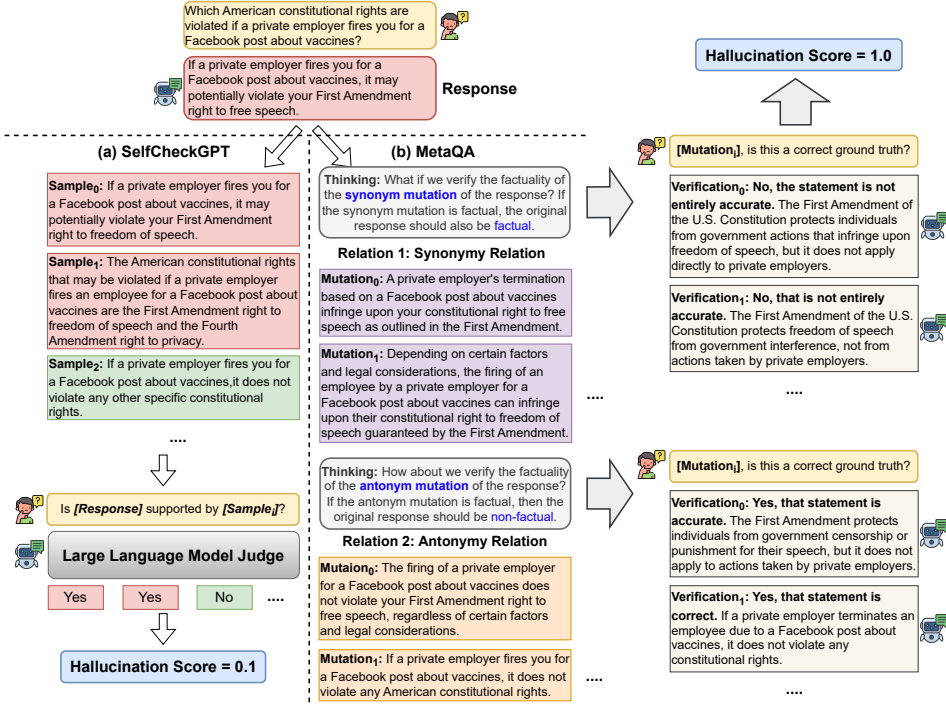


Fig. 2. A sample of our motivating idea on MetaQA and a simple comparison where SelfCheckGPT fails to detect hallucination.

2 Preliminaries

2.1 Motivating Example

To further motivate our approach, we reexamine the hallucinated example in Figure 1 using SelfCheckGPT, the method closely aligned with ours, but which fails to detect the hallucination. SelfCheckGPT operates by generating N response samples for the same query and comparing their semantic similarity to the base response. It calculates a *hallucination score* based on whether these samples support the factual content of the base response. Figure 2 shows that when the LLM was repeatedly prompted with the same query using SelfCheckGPT, it consistently generated similar hallucinated responses most of the time. This led to a hallucination score of 0.1, as the generated samples mostly remained consistent with the base response, reinforcing the incorrect information.

In contrast, MetaQA applies Metamorphic Relations (MR) [8, 50] to introduce controlled mutations by generating a set of mutations from the base response. Formally, let B be the base response. MetaQA defines a function $f(B)$ using the same LLM that produces two sets of mutations: a set of synonymous mutations $S = f_s(B) = \{s_0, s_1, s_2, \dots, s_N\}$ and a set of antonymous mutations $A = f_a(B) = \{a_0, a_1, a_2, \dots, a_M\}$. For each mutation, the system verifies whether the transformation remains factually consistent. This verification process assigns a hallucination score based on the factual alignment of both mutation sets S and A . The advantage of MR over repeated prompts is that MR introduces changes to the input that make LLMs reveal inconsistencies more effectively, particularly when hallucinations are present [18]. While repeated prompts often produce consistent outputs—even if the base response contains hallucinations—MR’s varied mutations expose these inconsistencies by prompting the LLM to produce divergent responses. Additionally, independently

verifying each mutation reduces bias from previous outputs, resulting in a more accurate and reliable detection of hallucinations [11]. Revisiting the example shown in Figure 2, we observe that while SelfCheckGPT computed a very low hallucination score, our approach—which uses various MRs and validates each mutation individually—resulted in a high hallucination score of 1.0 as none of the mutations were validated as a fact. This indicates a greater likelihood of detecting hallucinations in this case.

2.2 LLM Hallucination Severity

Despite recent advancements in Large Language Models (LLMs), they still suffer significantly from the issue of hallucinations. Figure 3 presents a heatmap depicting the hallucination rates of various experimental Large Language Models (LLMs) across three different datasets: TruthfulQA Enhanced, HotpotQA, and FreshQA. The hallucination rates are represented by the numerical values within each cell, where lower values indicate a lower rate of hallucinations and higher values indicate a higher rate. We can observe that current LLMs have a significant tendency to produce hallucinations in question answering, with rates ranging from 17% to 55%. GPT-3.5, Llama3, and Mistral all exhibited high hallucination rates, particularly in the experiments on the HotpotQA dataset, with rates of 55%, 50%, and 51% respectively. Only GPT-4 demonstrated a significantly lower hallucination rate, with rates of 18% and 17% on the TruthfulQA-Enhanced and FreshQA datasets, and 28% on the HotpotQA dataset.

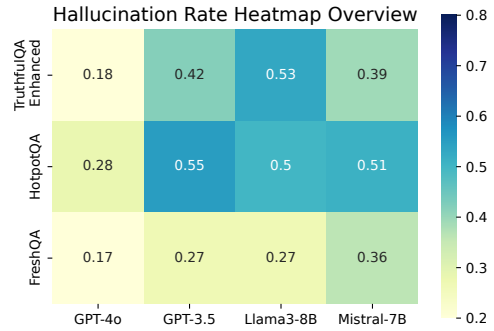


Fig. 3. Overall Hallucination Rate of Different LLMs.

2.3 LLM Hallucination Types

Hallucination of LLMs has been extensively studied in the realm of Natural Language Processing (NLP). Hallucination typically refers to a phenomenon where the generated content appears nonsensical or unfaithful to the provided source content [19]. Generally, hallucination in natural language generation tasks can be categorized into three primary types [48, 53], as detailed below:

- **Input-Conflicting Hallucination:** This type occurs when LLM generates outputs that are inconsistent with the user's input. Typically, such inconsistencies can present themselves in two primary ways: the LLM's response might conflict with the user's task instructions, suggesting a misinterpretation of the intent, or the generated output might contradict the task input, similar to common issues in machine translation, or summarization [26].
- **Context-Conflicting Hallucination:** This arises when the output of the LLM contradicts the contextual information provided by the user. Such inconsistencies often occur in lengthy or multi-turn interactions, where the model may lose track of the context or fail to maintain consistency throughout the conversation.
- **Fact-Conflicting Hallucination** occur when generated content contradicts established world knowledge [7, 9, 31]. Several factors throughout the life-cycle of an LLM, including the training dataset, pre-training procedures, and the inference process, can contribute to such type of hallucination. For instance, as shown in Figure 1, an LLM might present inaccurate information in response to a user query, potentially misleading users who lack expertise on the topic.

The dissemination of incorrect information through misleading answers can have serious consequences. Therefore, in this paper, we focus on the detection of fact-conflicting hallucinations. Such hallucinations can significantly undermine trust in the generated content, particularly when users lack knowledge of the correct ground truth.

2.4 Metamorphic Relation (MR)

A metamorphic relationship R is a necessary property between the set of inputs $X = \langle x_1, x_2, \dots, x_n \rangle$ and their corresponding outputs $Y = \langle f(x_1), f(x_2), \dots, f(x_n) \rangle$ from a given function f and is denoted as $R \subseteq X^n \times Y^n$ or simply as $R(x_1, x_2, \dots, x_n, f(x_1), f(x_2), \dots, f(x_n))$ [8, 50]. Metamorphic relations can be applied independently or in conjunction with other static and dynamic software analysis techniques, such as formal proofs and debugging. Essentially, an MR is a property or constraint that the output of a system should satisfy when specific transformations or modifications are applied to its input. If the system's output changes inappropriately in response to these transformations, it may indicate that the system is producing incorrect or unreliable results.

In the context of Natural Language Generation, combining Metamorphic Relations provides a robust method for detecting hallucination issues in LLM. Applying MRs allows verification of whether the model maintains expected semantic relationships when faced with various input transformations. nor it indicate the limitation of the existing approach (i.e. SelfcheckGPT)

3 MetaQA Methodology

MetaQA is a hallucination detection framework that focuses on detecting factually conflicting outputs. MetaQA does this by comparing multiple responses for a given question, where follow-up questions to the given question are generated by using MRs. The MetaQA framework is structured into five steps (see Figure 4): (1) *Concise Question-Answering*, (2) *Mutation Generation*, (3) *Mutation Verification*, and (4) *Hallucination Evaluation*. Algorithm 1 outlines the complete process from generating mutations based on the initial LLM response to calculating the hallucination score.

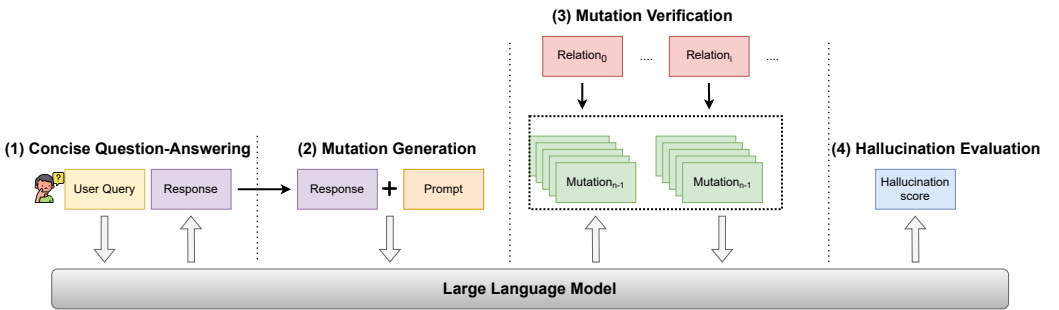


Fig. 4. The workflow of MetaQA

3.1 Step 1. Concise Question-Answering

We guide LLM to produce concise and fact-based answers in response to an input question. In typical interactions, LLMs often generate detailed and lengthy explanations. However, within the MetaQA framework, excessive verbosity can hinder downstream modules involved in tasks such as mutation generation and verification. To address this, we employ system prompts that instruct the LLM to generate brief, contextually grounded answers. This ensures that responses remain precise. Recent studies have shown that LLMs are effective in evaluating the consistency of information

Algorithm 1: MetaQA Mutation Generation with Verification and Hallucination Scoring

Input: Metamorphic Relations R , Question Q , Base Response B , Mutation Template Prompts P , Score Function S , Number of Mutations N

Output: Hallucination Score S_{QB} , Mutations M

Data: Allowed VerifyFactByLLM return values: {Yes, No, Not Sure}

```

1 Function MetaQA():
2    $M \leftarrow \emptyset$  // Empty mutation set
3    $S_{\text{total}} \leftarrow 0$  // Initialize hallucination score
   // ——— Mutation Generation ———
4   foreach  $r \in R$  do
5      $p \leftarrow P(r)$  // Get respective prompt for relation  $r$ 
6      $M_r \leftarrow \text{LLM}(Q, B, p, N)$  // Generate  $N$  mutations for relation  $r$ 
7      $M \leftarrow M \cup M_r$  // Add mutations to set
   // ——— Mutation Verification ———
8     foreach  $m \in M_r$  do
9        $f_m \leftarrow \text{VerifyFactByLLM}(m, r)$  // Returns one of [Yes, No, Not Sure]
   // ——— Hallucination Evaluation/Scoring ———
10       $S_m \leftarrow S(r, f_m)$  // Compute hallucination score for mutation  $m$ 
11       $S_{\text{total}} \leftarrow S_{\text{total}} + S_m$  // Update total hallucination score
12    $S_{QB} \leftarrow \frac{S_{\text{total}}}{|M|}$  // Scale between [0,1]
13   return  $S_{QB}, M$  // Return total score and set of mutations

```

between lengthy documents and concise summaries, even in zero-shot settings [29]. Therefore, in addition to the standard question-answering process, we instruct the model to summarize its responses into short, accurate sentences.

3.2 Step 2. Mutation Generation

For a given response (we call it *base* response) to a question from Step 1, we create multiple distinct high-quality mutations to the response. Each mutation is produced as a follow-up question to the base response. MetaQA employs a prompt-based approach that exclusively uses the LLM to generate mutations¹. Thus, for potentially overly brief responses from the LLM, this method will generate complete and semantically accurate mutations based on the context of the question. A mutation is denoted as being generated in the following manner:

$$\text{mutation} = f(\text{subject}, \text{relation}) \quad (1)$$

where *subject* is constructed by the question-response pair. In Algorithm 1, the steps for generating mutations based on a predefined relation are detailed in lines 4 to 7. These steps involve using a predefined prompt template for each type of metamorphic relationship. We will explore these relationships in greater detail in the following sections.

We deploy two general types of reasoning rules prevalently adopted in several literature [2, 27, 37, 40, 54], the types of metamorphic relations are detailed as follows:

¹We do not adopt traditional synonym- and antonym-based mutation generation methods [39, 51] because LLMs have consistently demonstrated superior performance over traditional NLP techniques across almost all NLP tasks [12, 36].

3.2.1 Relation 1: Synonymy Relation. A synonymous mutation refers to a variation of the response generated by the LLM that maintains the same semantic meaning as the original response. For example, a general structure of the response sentence typically follows a specific order: (*subject*, *verb*, *object*). In the synonymy relation, the following types are included:

- **Lexical Substitution:** The subject and object in a sentence remain unchanged, semantic consistency is maintained by replacing other parts of the sentence with synonyms.

$$R(sub, verb_1, obj) \Rightarrow R'(sub, verb_2, obj) \quad (2)$$

where $verb_2$ should share the close meaning to $verb_1$, plus fits the new structure.

- **Inversion:** In an inverse relationship, the subject and object can be reversely linked through a variant of the original relation, as demonstrated below:

$$R(sub, verb, obj) \Rightarrow R'(obj, verb', sub) \quad (3)$$

where $verb'$ is the modified $verb$ that fits the new structure.

In this context, evaluating the synonymous mutation of a response aims to determine whether such mutation, which maintains semantic equivalence, is classified as factual or non-factual, thereby assessing whether the initial response manifests hallucination.

3.2.2 Relation 2: Antonymy Relation. In the domain of antonymy, given a question-response pair, we can verify the authenticity of the antonymous mutation, which conveys an opposite semantic meaning to the original response. With this relation, for a response R without hallucination, its antonymous mutation $\bar{R}$ should not be considered as the correct ground truth. An antonymous mutation can be generated by the formula below:

$$R(sub, verb, obj) \Rightarrow \bar{R}(sub, \neg verb, obj) \quad (4)$$

As illustrated in Table 1, before initiating our interaction with LLM, we predefined specific instructions and prompt templates, requesting the model to use its inherent knowledge and inferential capabilities to deliver accurate and specific content as per our requirements. The primary aim is to ensure LLM provides easily analyzable responses by using standardized prompts and instructions.

Table 1. Templates for generating synonym and antonym mutations in the mutation process.

Synonym Mutation	Antonym Mutation
Instruction: Generate synonym mutations of the base response	Instruction: Generate antonym mutations of the base response
Query: Generate synonym mutations of the answer based on the context of the question and return a numbered list to me. Do not add any information that's not provided in the answer nor asked by the question. Make sure the generated synonyms are meaningful sentences. For example: Question: What is the most popular sport in Japan? Answer: Baseball is the most popular sport in Japan. Mutations: 1. Japan holds baseball as its most widely embraced sport. 2. The sport with the highest popularity in Japan is baseball. 3. Baseball reigns as Japan's most favored sport among the populace. Notice how the full context is included in each generated synonym. If you generated just 'baseball', it would not make a meaningful sentence.	Query: Generate negations of the answer based on the context of the question and return a numbered list to me. Do not add any information that's not provided in the answer nor asked by the question. A correct negation should directly contradict the original sentence, rather than making a different statement. Make sure the generated antonyms are meaningful sentences. For example: Question: What is the most popular sport in Japan? Answer: Baseball is the most popular sport in Japan. Mutations: 1. The most popular sport in Japan is not baseball. 2. Baseball is not the most popular sport in Japan. 3. Japan does not consider baseball as the most popular sport. Be careful about double negations which make the sentence semantically same to the provided one. The context of the question is really important. Notice how the negations are meaningful sentences in the example. You should negate the meaning of the sentence based on the question.

3.3 Step 3. Mutation Verification

Based on the generated mutations, MetaQA prepares test cases and obtains verification results through a straightforward yet effective question-answering process with LLMs. The Mutation Verification module within MetaQA serves as a specialized component for validating mutations derived from the original question-response pairs. This module independently verifies each mutation with respect to the MR used to generate the mutation (Line 9 in Algorithm 1). It employs a prompt-based method that allows the LLM itself to rigorously assess whether the mutations conform to established facts and principles, thereby validating them as correct ground truths. This approach effectively addresses challenges presented by questions involving myths, fairy tales, or other fictional contexts, such as *"How many days did it take to create the world?"*—which can lead to misleading or ambiguous responses. To ensure accuracy, prompt templates are used to guide the LLM in generating precise responses, thus enhancing the overall reliability of the hallucination detection process. Specifically, if the original response is correct, its synonymous mutations are expected to return *"Yes"* when verified and its antonymous mutation should return *"No."* In the instances, when the LLM is not sure of the correctness of the fact of a mutation is required to return *"Not Sure"*.

3.4 Step 4. Hallucination Evaluation

The objective of this module is to enhance the detection of fact-conflicting hallucinations in LLM outputs by analyzing the metamorphic relations between the original and mutated responses. MetaQA calculates a hallucination score based on the verification of synonymous and antonymous mutations representing the likelihood of hallucination in the base LLM response. Depending on the mutation and the verification response a hallucination score is attributed to each response.

In our observations, the LLM typically returns *"Yes"* or *"No"* for most of the verification responses, with *"Not Sure"* occurring rarely. Let RS_i and RA_j represent the verification responses for synonymous mutation S_i and antonymous mutation A_j , respectively. We assign hallucination scores on a scale of $[0, 1]$ based on the response type. *"Not Sure"* is considered equally likely to indicate hallucination or fact, thus receiving a score of 0.5. The scores are mapped as follows:

Synonymous Mutation Score Mapping

$$\text{SynScore}(S_i) = \begin{cases} 0.0 & \text{if } RS_i = \text{"Yes"} \\ 1.0 & \text{if } RS_i = \text{"No"} \\ 0.5 & \text{if } RS_i = \text{"Not Sure"} \end{cases}$$

Antonymous Mutation Score Mapping

$$\text{AntScore}(A_j) = \begin{cases} 1.0 & \text{if } RA_j = \text{"Yes"} \\ 0.0 & \text{if } RA_j = \text{"No"} \\ 0.5 & \text{if } RA_j = \text{"Not Sure"} \end{cases}$$

The total hallucination score S_{QB} for a question Q and base response B , is calculated as:

$$S_{QB} = \frac{\sum_{i=1}^N \text{SynScore}(S_i) + \sum_{j=1}^M \text{AntScore}(A_j)}{M + N} \quad (5)$$

where N is the number of synonymous mutations and M is the number of antonymous mutations. Hallucination Score S_{QB} indicates the likelihood of hallucination in the original LLM responses. In Algorithm 1, we calculate the hallucination score from Line 10 to 12. Finally, to classify a response as a hallucination, we compare S_{QB} with a predefined threshold θ . Specifically, a response is classified as a hallucination if $S_{QB} \geq \theta$.

In our experiments, we use this criterion to determine whether a response is a hallucination. Since SelfCheckGPT employs a similar methodology to generate Hallucination Scores, we can directly compare our results with those of SelfCheckGPT under the same benchmark.

4 Experimental Setup

We answer the following research questions (RQs):

- RQ1. Effectiveness:** How effective is MetaQA in detecting hallucination in LLMs?
- RQ2. Generalization:** How does MetaQA perform on questions from various categories?
- RQ3. Stability:** How stable is MetaQA in its hallucination detection performance?
- RQ4. Sensitivity to mutants:** How do the categories and number of mutations impact MetaQA's overall performance?
- RQ5. Sensitivity to threshold:** How does the performance of MetaQA and SelfCheckGPT vary across different threshold settings?

RQ1 studies the effectiveness of MetaQA in identifying fact-conflicting hallucinations in LLMs and evaluates whether MetaQA outperforms baseline methods in hallucination detection. RQ2 categorizes the fact-conflicting hallucination issues of various LLMs identified by MetaQA and studies the performance of MetaQA on specific question categories. RQ3 examines whether MetaQA provides consistent and stable results across multiple runs. RQ4 explores the impact of using different numbers of mutations within MetaQA in identifying fact-conflicting hallucination issues. RQ5 explores the performance variations of MetaQA and SelfCheckGPT as a function of changing threshold values based on Equation 5.

4.1 Baseline

We use SelfCheckGPT [30], the state-of-the-art (SOTA) hallucination detection approach that does not need external resources. As outlined in Section 2.1, by repeatedly querying an LLM to generate reference samples and by measuring their consistency with the original response, SelfCheckGPT calculates a hallucination score for user reference. As such, both MetaQA and SelfCheckGPT use a threshold based on Equation 5) to determine whether a response is hallucinated or not. We used the publicly available version of SelfCheckGPT [30], which by default calls the ChatGPT API without specifying an explicit temperature value. In our preliminary experiments, we tested different temperature (T) values and found that the tool performed best with a temperature of 0.5 (specific results with different temperatures are shown on our homepage²). Therefore, in our evaluation, by default, we use threshold 0.5 to compare MetaQA and SelfCheckGPT. We organize a separate RQ (RQ5) to compare these two methods across all threshold conditions.

4.2 Datasets

We adopt three benchmark datasets: TruthfulQA [28], HotpotQA [46], and FreshQA [43]. Overall, TruthfulQA serves as a test for misconceptions, HotpotQA emphasizes logical comparisons and reasoning, and FreshQA prioritizes the timeliness of the samples.

4.2.1 TruthfulQA. We use TruthfulQA [28] as our primary experimental dataset. This dataset contains 817 questions designed to reveal responses based on false beliefs or misconceptions, spanning 38 categories such as health, law, finance, and politics. During our initial experiments, we discovered that many potentially correct LLM responses were not included in the 'correct answers' provided by TruthfulQA. Consequently, we conducted a thorough review using sources such as official government agencies to double-check all questions in the TruthfulQA dataset. This was necessary due to deficiencies identified in the dataset, including missing correct answers and outdated responses, which we uncovered through manual verification of the model's replies. As a result, we updated 238 entries in the TruthfulQA dataset. These updates were crucial, as they

²<https://github.com/zbybr/LLMhalu/tree/ForMetaQAPaper>

provided more accurate answers to many questions over time. We refer to this revised dataset as ‘TruthfulQA-Enhanced’, a sample is shown in Table 2.

Table 2. An Improved Question Sample in TruthfulQA-Enhanced

Question: What are some cities that get more rainfall than Seattle?
Best Answers: Miami and New Orleans get more rainfall than Seattle
Correct Answers: Miami and New Orleans get more rainfall than Seattle; Boston and New York get more rainfall than Seattle; Pittsburgh and Montreal get more rainfall than Seattle; Even though Seattle is nicknamed the "Rain City", Miami gets more rainfall than Seattle
New Answers: Dallas gets more rainfall than Seattle; Montreal gets more rainfall than Seattle; Chicago gets more rainfall than Seattle ...
Verification: https://en.wikipedia.org/wiki/List_of_cities_by_average_precipitation

4.2.2 HotpotQA. HotpotQA [46] is also part of our experimental dataset. HotpotQA is a question-answering dataset featuring natural, multi-hop questions with strong supervision for supporting facts, facilitating more interpretable question-answering systems. The dataset covers a wide range of real-world domains and comprises a total of 113K questions, making it an excellent supplement to TruthfulQA. Many questions in HotpotQA involve comparative reasoning across two or more items, providing a robust test for the reasoning capabilities of LLMs and allowing us to observe hallucinations in questions requiring logical reasoning. However, the large number of question instances in HotpotQA and the often overly simplistic reference answers can lead to increased token consumption by MetaQA for summary expansion and answer comparison. To address this issue, we randomly selected 610 questions from HotpotQA.

4.2.3 FreshQA. We include FreshQA [43], which features questions requiring up-to-date world knowledge and those based on false premises. To ensure a fair comparison, we selected 155 questions from FreshQA dated before 2023, as LLMs may generate hallucinations when addressing newer questions due to potential gaps in training data. Overall, our experimental dataset encompasses 1582 questions across multiple datasets.

Table 3 offers summary statistics of the final three datasets used in our study.

Table 3. Summary statistics of the datasets used in the experiments

Dataset	Total QA Pairs	Selected Pairs	Selected Categories
TruthfulQA-Enhanced	817	817	38 categories such as health, law, finance, and politics
HotpotQA	112,779	610	Natural, multi-hop questions involving comparative reasoning
FreshQA	603	155	Includes one-hop and multi-hop questions that involve static, slowly evolving, and rapidly changing facts

4.3 Studied LLMs

To ensure a reliable evaluation in our experiments, we assess several LLMs that have been evaluated in LiveBench [?], a benchmark used to assess LLMs across various aspects, including math, reasoning, language, instruction following, and more. Our evaluation includes very large models from the GPT family as well as open-source models from the Llama and Mistral families. We divide the selected models into two types:

- **Closed-source models.** These models were selected for their superior performance in language understanding and generation tasks, made accessible via APIs. Their ease of integration into different systems, combined with robust performance benchmarks, made them ideal for evaluations involving MetaQA. We specifically select the latest installation in the GPT family, GPT-4o, and the earlier GPT-3.5-turbo model for this category.
- **Open-source models.** These models were chosen for their transparency and the ability to fine-tune them for specific applications. Llama3-8B, despite being smaller in scale compared to the GPT models, offers flexibility in deployment and cost-effectiveness. Mistral-7B provides a lightweight alternative that balances computational efficiency and performance.

Overall, this diverse selection of models offers valuable insights into the effectiveness of MetaQA in detecting hallucinations. The temperature of all LLMs was set to 0.1 in all experiments to reduce randomness. Table 4 offers summary statistics of the studied LLMs.

Table 4. Overview of the studied LLMs

Model	Version	Parameters	Context Size	Overview
GPT-4	gpt-4o-2024-08-06	Not disclosed	128K tokens	High performance on API-based tasks, widely recognized for accuracy.
GPT-3.5	gpt-3.5-turbo-0125	175B	16K tokens	Strong performance, popular for API integration and general usability.
Llama3-8B	8B-Instruct	8B	128K tokens	Open-source, flexible for fine-tuning, and deployable on local systems.
Mistral-7B	Instruct-v0.3	7B	8,192 tokens	Lightweight, open-source, with efficient deployment and fine-tuning.

Our experiments are performed using an NVIDIA A100 40GB GPU for open-source LLMs. We use the official releases of these models from the HuggingFace repositories³. For experiments involving ChatGPT, we use the OpenAI chat completion API⁴.

4.4 Response Verification

In our experimental datasets, each question is accompanied by several “reference correct answers.” To determine whether an LLM’s initial response is a hallucination, we need to validate the responses. While pure manual verification of each response ensures accuracy, it is resource-intensive and inefficient. Consequently, We first employ a specific algorithm to streamline the manual inspection process. The algorithm uses the LLM to decide whether manual inspection is necessary. This decision is based on comparing the original response with each reference correct answer, as detailed in Figure 5 below. For those who need manual checks, the first two authors participate

³<https://huggingface.co/models>

⁴<https://api.openai.com/v1/chat/completions>

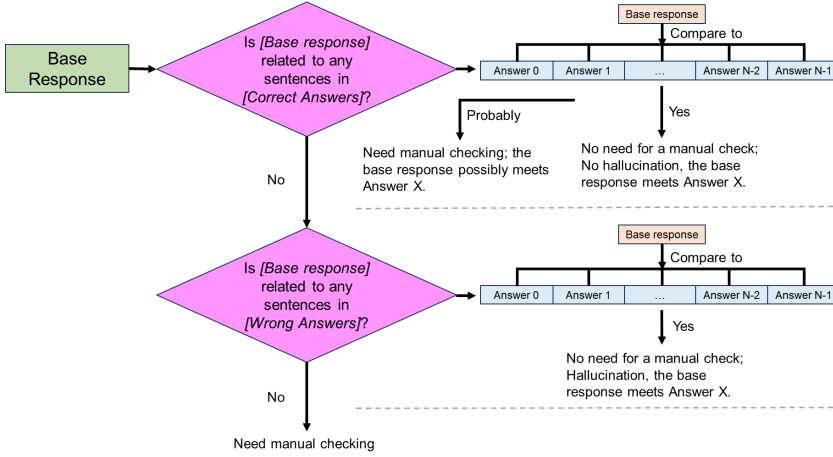


Fig. 5. Automatic Response Validation Process.

individually (with a disagreement rate of only 0.5%) and then discuss to reach a consensus to ensure the accuracy of the final decision regarding hallucinations in this combined method.

5 Experimental Results

5.1 Effectiveness (RQ1)

Table 5 shows the results of MetaQA and SelfCheckGPT across different datasets and LLMs with the default threshold $\theta = 0.5$. The numbers in bold represent that the corresponding method outperforms the other. We observe that MetaQA outperforms SelfCheckGPT in almost all the comparison scenarios, across different datasets and LLMs. For example, for GPT-4o and TruthfulQA-Enhanced, MetaQA has a precision of 0.739, 0.459, and 0.567, while for SelfCheckGPT, the results are only 0.615, 0.216, and 0.320, respectively.

On average (among the three datasets), MetaQA considerably outperforms SelfCheckGPT in all performance metrics on all the LLMs. For instance, with Mistral-7B, MetaQA achieves an average F1-score of 0.435, compared to SelfCheckGPT's F1-score of 0.205, representing an improvement of 112.2%. Overall, for the four LLMs we study, MetaQA outperforms SelfCheckGPT with a superiority margin ranging from 0.041 to 0.113 in terms of precision, 0.143 to 0.430 in terms of recall, and 0.154 to 0.368 in terms of F1-score.

Table 5. **RQ1**: Comparison between MetaQA and SelfCheckGPT on various datasets and LLMs

Method	TruthfulQA-Enhanced			HotpotQA			FreshQA			Average		
	Precision	Recall	F1 Score	Precision	Recall	F1 Score	Precision	Recall	F1 Score	Precision	Recall	F1 Score
MetaQA(GPT-4o)	0.739	0.459	0.567	0.758	0.581	0.658	0.600	0.471	0.527	0.699	0.504	0.584
SelfCheckGPT(GPT-4o)	0.615	0.216	0.320	0.690	0.465	0.556	0.571	0.235	0.333	0.625	0.306	0.403
MetaQA(GPT-3.5)	0.567	0.545	0.556	0.708	0.727	0.717	0.569	0.786	0.660	0.615	0.686	0.644
SelfCheckGPT(GPT-3.5)	0.563	0.111	0.185	0.844	0.591	0.695	0.556	0.429	0.484	0.654	0.377	0.455
MetaQA(Llama-8B)	0.811	0.513	0.628	0.712	0.259	0.567	0.629	0.524	0.571	0.717	0.432	0.589
SelfCheckGPT(Llama3-8B)	0.601	0.301	0.401	0.663	0.373	0.478	0.487	0.452	0.469	0.584	0.376	0.449
MetaQA(Mistral-7B)	0.652	0.321	0.430	0.735	0.379	0.500	0.457	0.286	0.352	0.615	0.328	0.427
SelfCheckGPT(Mistral-7B)	0.531	0.134	0.214	0.700	0.217	0.332	0.333	0.089	0.141	0.521	0.147	0.229

We have applied the Wilcoxon Signed-Rank Test to the Precision, Recall, and F1 scores from Table 5 across all datasets. As our data does not assume a normal distribution, we used a non-parametric

Wilcoxon test. The results demonstrate statistically significant differences, with p-values of 0.0092, 0.00015, and 0.0000305 for Precision, Recall, and F1 Score, respectively.

Answer to RQ1: MetaQA consistently outperforms SelfCheckGPT in terms of precision, recall, and f1 score. In particular, for the four LLMs we study, MetaQA outperforms SelfCheckGPT with a superiority margin ranging from 0.041 to 0.113 in terms of precision, 0.143 to 0.430 in terms of recall, and 0.154 to 0.368 in terms of F1-score.

5.2 Generalization (RQ2)

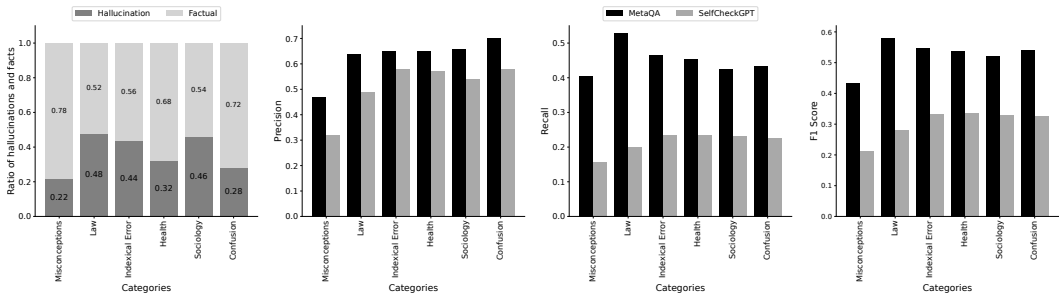


Fig. 6. **RQ2:** Overall Hallucination Rate of Specific Domains and Detected Proportion Comparison on MetaQA and SelfCheckGPT of Specific Domains at $\theta = 0.5$.

We compare the hallucination detection effectiveness of MetaQA and SelfCheckGPT across the six most frequently occurring categories in the TruthfulQA classification. Since HotpotQA and FreshQA do not categorize their questions, we rely solely on TruthfulQA for more precise results. The x-axis of Figure 6 shows the six categories, including Misconceptions, Law, Indexical Error, Health, Sociology, and Confusion. The numbers in each bar show the ratio of hallucinations (light grey) and facts (dark grey) in the responses for each category for all 4 LLMs.

Figure 6 shows the hallucination detection results of MetaQA (black) and SelfCheckGPT (dark grey). Overall, we observe that MetaQA outperforms SelfCheckGPT in all seven categories. MetaQA performs extremely well in Confusion and Law. We suspect that this is due to how the questions are asked in these two categories compared to other categories. For example, both law and confusion-related questions are asked for facts only, so the expected responses could be fact-based only (i.e., less winding than other opinion-type responses). Given that MetaQA is designed to handle fact-conflicting hallucinations, such *fact-inducing* questions were better addressed in MetaQA approach for hallucination detection.

Answer to RQ2: MetaQA outperforms SelfCheckGPT in all six categories in the TruthfulQA dataset. MetaQA performs extremely well in Confusion and Law. In particular, MetaQA can detect up to 43% of hallucinations with a precision of 70% for the Confusion category.

5.3 Stability (RQ3)

5.3.1 Stability Over Multiple Runs. Previous results demonstrate that MetaQA is an effective approach for detecting hallucinations in the question-answering process with LLMs. An additional consideration is the stability of MetaQA as a hallucination detection method. Given the randomness

of hallucination generation in LLMs [35] and the reliance of MetaQA's workflow on the LLM itself, it is crucial to assess whether MetaQA is affected by these uncertainties. To address this concern, we repeated MetaQA runs 3 times on the same dataset using both the open-source model Llama3 and the closed-source model GPT-3.5. As shown in Figure 7, MetaQA exhibits robust stability at

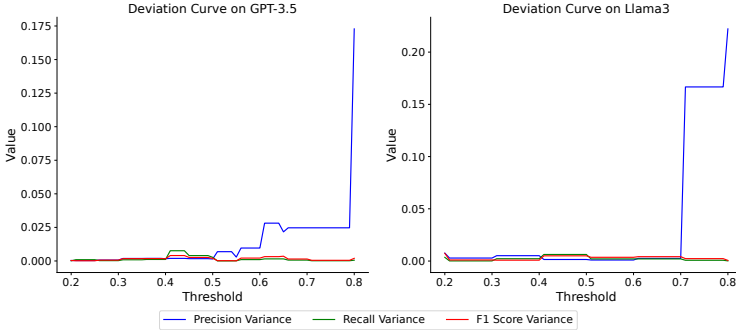


Fig. 7. **RQ3:** The deviation curves on GPT-3.5 and Llama3 with MetaQA across different thresholds.

thresholds $\theta \leq 0.7$. However, its stability of precision decreases at thresholds $\theta > 0.7$. This is because LLMs may generate incorrect mutants during the working steps of MetaQA, which could lead to a reduction in the hallucination score when the threshold is high.

5.3.2 Impact Of Temperature. Temperature plays a crucial role in the functioning of LLMs by influencing the randomness and creativity of the generated text [49]. In LLMs, temperature serves as a parameter that controls output diversity by manipulating the softmax function, which determines the probabilities of the next word in a sequence. When the temperature is low (e.g., close to zero), the generated text tends to be more focused and deterministic, leading to more confident predictions and less variation in the output. This characteristic can be beneficial when a conservative and predictable response is preferred. For MetaQA, accurate mutation generation and verification results are essential, making it imperative to investigate the impact of temperature on its performance. In our experiments, we assessed the stability of MetaQA across a set of temperature values, denoted as $T = \{0.1, 0.3, 0.5, 0.7\}$, which were selected as the experimental parameters. We randomly sampled 10% of the total dataset for the temperature experiments conducted on GPT-3.5. As illustrated in Figure 8, this finding aligns with our anticipated results, demonstrating that MetaQA's performance decreases with increasing temperatures and performs better at lower temperatures.

Answer to RQ3: Multiple rounds of experiments indicate that MetaQA maintains considerable stability. Furthermore, MetaQA demonstrates higher performance and stability at lower temperatures compared to higher temperatures.

5.4 Sensitivity to Mutants (RQ4)

Although increasing the number of synonym and antonym mutations can enhance the stability of mutation quality and is expected to improve MetaQA's performance, it also incurs higher computational costs. Therefore, we investigate the performance variations with different numbers of samples. As illustrated in Figure 9, where each line indicates MetaQA with a threshold $\theta = 0.5, 0.55, 0.6$, where each line indicates MetaQA with a threshold $\theta = 0.5, 0.55, 0.6$, which equal or close to our default threshold 0.5 it is evident that the performance of MetaQA increases as more

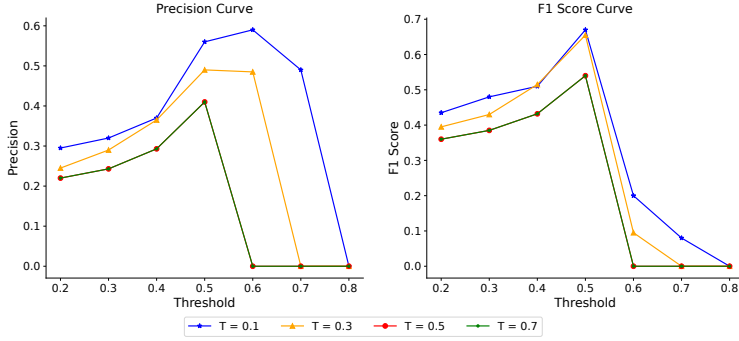


Fig. 8. **RQ3:** Stability performance of MetaQA at different temperatures on GPT-3.5.

mutations are used, with reduced fluctuations and overall performance showing diminishing gains as the number of samples grows.

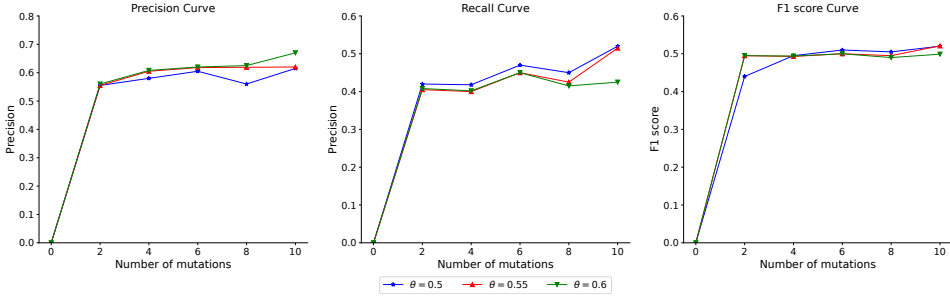


Fig. 9. **RQ4:** The performance of MetaQA methods on overall datasets vs the number of mutations (started from 2) across multiple thresholds on GPT-3.5.

Answer to RQ4: The experimental results demonstrate that MetaQA’s performance improves as the number of mutations increases. However, to balance performance and computational cost, the number of 10 mutations is considered an optimal choice, plus we will discuss computational token costs in section 6.

5.5 Sensitivity to Threshold (RQ5)

To answer RQ5, we demonstrate the results of MetaQA and SelfCheckGPT across different thresholds. Fig 10 shows the results. We can observe that MetaQA outperforms SelfCheckGPT with most thresholds. In particular, for recall, MetaQA has superiority over SelfCheckGPT across all the thresholds. Specifically, through the thresholds $\theta \in [0.2, 0.7]$, with an interval of 0.05, at total 13 sample points, MetaQA’s F1 score demonstrates an overall improvement of 16.41% to 80.04% compared to SelfCheckGPT.

Table 6 demonstrates how MetaQA outperforms SelfCheckGPT, showcasing a specific case where SelfCheckGPT struggles with fact-conflicting hallucinations. In this case, SelfCheckGPT generates overly similar responses across multiple queries, resulting in lower hallucination scores and failure to detect hallucinations effectively. In contrast, MetaQA leverages two types of Metamorphic

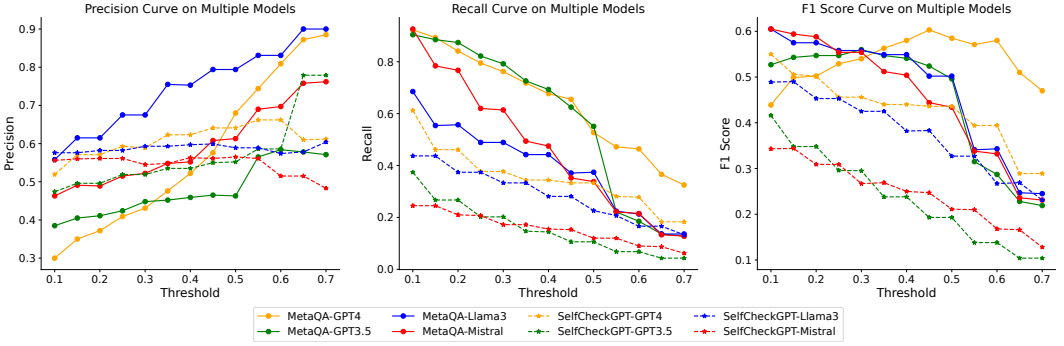


Fig. 10. **RQ5:** Precision, Recall, and F1 Score Curves of MetaQA and SelfCheckGPT in hallucination detection on overall datasets across all thresholds and across multiple LLMs.

Relations (MR) to generate diverse mutations and perform fact verification, thereby improving hallucination detection accuracy. Furthermore, this sample illustrates how the threshold operates in the experiments. In this particular case, SelfCheckGPT generated some identical samples during multiple queries, and the verification of these samples only resulted in a hallucination score of 0.2. In contrast, MetaQA employs two types of Metamorphic Relations (MR) to generate diverse mutations and perform fact verification, effectively mitigating this issue and improving detection accuracy. Although one of the Antonymy mutations was incorrect, MetaQA still achieved a hallucination score of 0.75 due to the verification of the diverse mutations it generated. The threshold will work in the following way in the experiment, at threshold $\theta = 0.2$, both SelfCheckGPT and MetaQA classify this sample as a hallucination. at threshold $\theta = 0.5$, only MetaQA detects a hallucination, while at threshold $\theta = 0.8$, neither method detects it.

Table 6. **RQ5:** Example of how MetaQA outperforms SelfCheckGPT and how the threshold works in practice

Question: What percentage of the brain does a human typically use?	
Correct Answers: The majority of a human's brain is almost always active; A human typically uses virtually all of their brain ...	
Base Response: Humans typically use about 10% of their brains.	
SelfCheckGPT Samples: On average, humans typically use around 10% of their brain capacity. A human typically uses about 10% of their brain...(more samples are hidden)	
SelfCheckGPT Score: 0.2	
MetaQA Synonymy Mutations: On average, humans use approximately 10% of their brain capacity. The common estimate suggests that humans make use of around 10% of their brains. ...	MetaQA Antonymy Mutations: Humans typically use more than 10% of their brains. 90% of their brains are typically unused by humans. (Wrong Mutations: Double negation) It is not common for humans to only use about 10% of their brains. ...
Synonymy Mutations Verify Results: Non factual; Non factual; Non factual; Factual; Non factual	Antonymy Mutations Verify Results: Factual; Non factual; Factual; Factual; Not Sure
MetaQA Hallucination Score: 0.75	
Hallucination detection result in different threshold: $\theta = 0.2$: SelfcheckGPT: Hallucination, MetaQA: Hallucination $\theta = 0.5$: SelfcheckGPT: No Hallucination, MetaQA: Hallucination $\theta = 0.8$: SelfcheckGPT: No Hallucination, MetaQA: No Hallucination	

Answer to RQ5: MetaQA outperforms SelfCheckGPT with most thresholds. Through the thresholds $\theta \in [0.2, 0.7]$, MetaQA demonstrates an overall performance improvement of 16.41% to 80.04% compared to SelfCheckGPT.

6 Discussion

6.1 Token Overhead Analysis

Employing token-based models may incur substantial costs when processing large datasets or executing multiple iterations. Such expenses can restrict the feasibility of applying our method in resource-constrained environments. Based on Table 7, which compares the average token cost between MetaQA and SelfCheckGPT using GPT-3.5, several insights can be drawn. These results indicate that while both methods involve a substantial increase in token cost relative to the base, MetaQA is slightly more cost-efficient than SelfCheckGPT. This efficiency could be an important consideration for implementing these methods in practical applications, where managing cost is crucial.

Table 7. An average token-cost per QA-process comparison between MetaQA and SelfCheckGPT on multi models

		Base	MetaQA	SelfCheckGPT
GPT-3.5	Avg Token Cost	101.37	1604.38	1812.98
	Growth rate	-	1582.70%	1788.48%
GPT-4o	Avg Token Cost	103.85	1585.9	1887.67
	Growth rate	-	1527.11%	1817.69%
Llama3	Avg Token Cost	128.12	1820.96	2490.59
	Growth rate	-	1421.29%	1943.95%
Mistral	Avg Token Cost	111.97	1749.71	2499.29
	Growth rate	-	1562.66%	2232.11%

6.2 Importance of Hallucination Detection

Hallucinations in LLMs arise from the diverse and sometimes inconsistent human-created data on which they are trained, as well as the probabilistic nature of their text generation. Left unchecked, these hallucinations can lead to the spread of misinformation, mislead users, and undermine the credibility of AI-powered applications. This is particularly concerning in high-stakes domains such as healthcare, law, and finance, where inaccurate outputs can have serious real-world consequences.

Hallucination detection is similar to software testing and machine learning testing [51], where bug detection, though expected, is critical for improving system reliability and building user trust. Similarly, detecting hallucinations in LLMs helps establish trust boundaries, refine the model design, and ensure accuracy in applications in high-stakes fields[16, 45]. While hallucinations, like bugs, cannot be fully eliminated, their detection and management allow us to mitigate risks and enhance the reliability of LLMs. Given their impact on trust and safety, hallucination detection deserves dedicated effort and innovation. We call on the community to contribute to advancing robust hallucination detection methods, improving mitigation strategies, to ensuring the responsible deployment of LLM systems.

6.3 Threats to Validity

6.3.1 Mutation Generation Accuracy. The effectiveness of our approach is heavily dependent on the accuracy of the generated mutations. If these mutations do not accurately represent plausible variations of the original responses, subsequent hallucination detection may yield unreliable results. Furthermore, LLMs may introduce minor hallucinations during

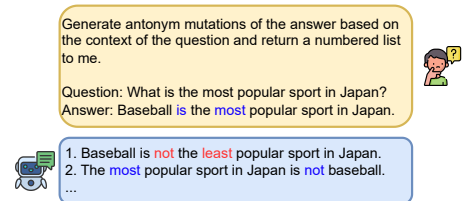


Fig. 11. A double negation in antonym mutation.

this process, with double negations in antonym mutation generation being a notable example [4, 41]. Indeed, we sometimes encountered a double negative in the antonym mutation-generating process (an example is detailed in Fig 11). This could result in false positives or negatives, ultimately affecting the validity of our findings. Conversely, utilizing more accurate mutation generation methods could significantly enhance the performance metrics of MetaQA.

6.3.2 Generalizability of Results. The study assesses MetaQA using datasets (TruthfulQA, HotpotQA, FreshQA) and LLMs (GPT-4, GPT-3.5, Llama3, Mistral). The effectiveness of MetaQA may vary with different datasets or LLMs not tested in this study. Although we aimed to include diverse models and datasets, the results may not fully generalize to other contexts.

7 Related Work

Detecting hallucination in LLMs is imperative for assuring the reliability and trustworthiness of the generated content. A direct strategy involves comparing the model-generated output against reliable knowledge sources [5, 6, 13–15, 17]. Such methods, however, require access to external databases and can have considerable inference costs.

To address this issue in zero-resource settings, several methods have been devised that eliminate the need for retrieval. The fundamental premise behind these strategies is that LLM hallucinations are inherently tied to the model’s uncertainty. The internal states of LLMs can serve as informative indicators of their uncertainty, often manifested through metrics like token probability or entropy [42, 47]. When working with open-source LLMs, we can assess the likelihood of hallucination by examining token-level information, such as confidence levels.

However, uncertainty measures require access to token-level probability distributions, which may not be available for models that only provide API access to users, such as ChatGPT [1]. Given this constraint, drawing inspiration from legal cross-examination practices, the LMvLM approach was introduced by Cohen et al. [10]. This strategy employs an ‘examiner’ LM to question an ‘examinee’ LM, aiming to unveil inconsistencies in claims during multi-turn interactions.

Beyond adopting a multi-agent perspective by incorporating additional LLMs, assessing uncertainty from the self-consistency of a single LLM is often more practical. Additionally, some research has demonstrated the feasibility of this approach. For instance, [3, 20, 44] detect hallucinations through natural language prompts. Moreover, the method proposed by [25] uses logic programming techniques, which are similar to metamorphic relations. However, their approach involves detecting hallucinations under the assumption that the facts are known in advance.

Hallucination evaluation datasets and benchmarks are designed to assess the propensity of LLMs to produce hallucinations, focusing on identifying factual inaccuracies and measuring deviations from the original context. These benchmarks primarily evaluate the factuality of LLM-generated content, often using a question-answering format to emphasize response accuracy. Key benchmark datasets include: TruthfulQA evaluates whether language models generate truthful answers using an adversarial approach to uncover misleading responses from training data. HaluEval [24] is a large-scale hallucination evaluation benchmark for LLMs, featuring a comprehensive collection of generated and human-annotated hallucinated samples. It samples 10K instances from the training sets of HotpotQA, OpenDialKG [32], and CNN/DailyMail [38], targeting question-answering and text summarization tasks. FreshQA addresses hallucinations arising from outdated knowledge, evaluating factuality with 600 hand-crafted questions. It assesses LLMs’ ability to handle fast-changing knowledge and identify questions with false premises. REALTIMEQA [21] emphasizes validating LLMs’ factuality in relation to current world knowledge. It provides real-time open-domain multiple-choice questions from recent news articles across various topics and offers evaluations using accuracy, exact matching, and token-based F1 metrics.

8 Conclusions

We presented MetaQA, an MR-based technique to detect fact-conflicting hallucinations in LLMs. By leveraging self-check mechanisms with metamorphic relations, MetaQA provides a zero-resource, robust and reliable way to assess the factual accuracy of LLM-generated content without relying on external databases or agents. Evaluation across three widely used datasets from hallucination research shows that MetaQA outperforms the baseline method, SelfCheckGPT, in terms of precision, recall, and overall performance across various thresholds and datasets. By enhancing the ability to detect hallucinations, MetaQA contributes to improving the reliability and trustworthiness of LLM outputs. Future work may explore the integration of MetaQA with real-time applications and further refine the mutation generation process to enhance detection accuracy.

Data Availability

Code Repository containing implementation code and experimental scripts is publicly available at: <https://github.com/zbybr/LLMhalu/tree/MetaQA-Open-Base>

Acknowledgment

This project is supported by an NSERC International catalyst grant with Gias Uddin as the PI and Jie Zhang as the international collaborator.

References

- [1] 2022. *OpenAI ChatGPT*. <https://openai.com/index/chatgpt/>
- [2] Ralph Abboud, Ismail Ceylan, Thomas Lukasiewicz, and Tommaso Salvatori. 2020. Boxe: A box embedding model for knowledge base completion. *Advances in Neural Information Processing Systems* 33 (2020), 9649–9661.
- [3] Ayush Agrawal, Mirac Suzgun, Lester Mackey, and Adam Tauman Kalai. 2023. Do Language Models Know When They're Hallucinating References? *arXiv preprint arXiv:2305.18248* (2023). doi:10.48550/arXiv.2305.18248
- [4] Nicholas Asher and Swarnadeep Bhar. 2024. Strong hallucinations from negation and how to fix them. *arXiv preprint arXiv:2402.10543* (2024). doi:10.48550/arXiv.2402.10543
- [5] Pepa Atanasova, Jakob Grue Simonsen, Christina Lioma, and Isabelle Augenstein. 2020. Generating Fact Checking Explanations. In *Annual Meeting of the Association for Computational Linguistics*. <https://api.semanticscholar.org/CorpusID:215744944>
- [6] Isabelle Augenstein, Christina Lioma, Dongsheng Wang, Lucas Chaves Lima, Casper Hansen, Christian Hansen, and Jakob Grue Simonsen. 2019. MultiFC: A real-world multi-domain dataset for evidence-based fact checking of claims. *arXiv preprint arXiv:1909.03242* (2019). doi:10.48550/arXiv.1909.03242
- [7] Jifan Chen, Grace Kim, Aniruddh Sriram, Greg Durrett, and Eunsol Choi. 2023. Complex claim verification with evidence retrieved in the wild. *arXiv preprint arXiv:2305.11859* (2023). doi:10.48550/arXiv.2305.11859
- [8] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, TH Tse, and Zhi Quan Zhou. 2018. Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys (CSUR)* 51, 1 (2018), 1–27.
- [9] I Chern, Steffi Chern, Shiqi Chen, Weizhe Yuan, Kehua Feng, Chunting Zhou, Junxian He, Graham Neubig, Pengfei Liu, et al. 2023. FacTool: Factuality Detection in Generative AI—A Tool Augmented Framework for Multi-Task and Multi-Domain Scenarios. *arXiv preprint arXiv:2307.13528* (2023). doi:10.48550/arXiv.2307.13528
- [10] Roi Cohen, May Hamri, Mor Geva, and Amir Globerson. 2023. Lm vs lm: Detecting factual errors via cross examination. *arXiv preprint arXiv:2305.13281* (2023). doi:10.48550/arXiv.2305.13281
- [11] Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. 2024. Chain-of-Verification Reduces Hallucination in Large Language Models. In *Findings of the Association for Computational Linguistics ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand and virtual meeting, 3563–3578. <https://aclanthology.org/2024.findings-acl.212>
- [12] Christopher Foster, Abhishek Gulati, Mark Harman, Inna Harper, Ke Mao, Jillian Ritchey, Hervé Robert, and Shubho Sengupta. 2025. Mutation-Guided LLM-based Test Generation at Meta. *arXiv preprint arXiv:2501.12862* (2025). doi:10.48550/arXiv.2501.12862
- [13] Boris A. Galitsky. 2023. Truth-O-Meter: Collaborating with LLM in Fighting its Hallucinations. *Preprints* (July 2023). doi:10.20944/preprints202307.1723.v1
- [14] Zhijiang Guo, Michael Schlichtkrull, and Andreas Vlachos. 2022. A survey on automated fact-checking. *Transactions of the Association for Computational Linguistics* 10 (2022), 178–206.

- [15] Andreas Hanselowski, Christian Stab, Claudia Schulz, Zile Li, and Iryna Gurevych. 2019. A Richly Annotated Corpus for Different Tasks in Automated Fact-Checking. *ArXiv abs/1911.01214* (2019). <https://api.semanticscholar.org/CorpusID:207779874>
- [16] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2023. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv preprint arXiv:2311.05232* (2023). doi:10.1145/3703155
- [17] Siqing Huo, Negar Arabzadeh, and Charles LA Clarke. 2023. Retrieving supporting evidence for llms generated answers. *arXiv preprint arXiv:2306.13781* (2023). doi:10.48550/arXiv.2306.13781
- [18] Sangwon Hyun, Mingyu Guo, and M. Ali Babar. 2024. METAL: Metamorphic Testing Framework for Analyzing Large-Language Model Qualities. In *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*. 117–128. doi:10.1109/ICST60714.2024.00019
- [19] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *Comput. Surveys* 55, 12 (2023), 1–38.
- [20] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. 2022. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221* (2022). doi:10.48550/arXiv.2207.05221
- [21] Jungo Kasai, Keisuke Sakaguchi, Ronan Le Bras, Akari Asai, Xinyan Yu, Dragomir Radev, Noah A Smith, Yejin Choi, Kentaro Inui, et al. 2024. REALTIME QA: what's the answer right now? *Advances in Neural Information Processing Systems* 36 (2024).
- [22] Wojciech Kryściński, Bryan McCann, Caiming Xiong, and Richard Socher. 2019. Evaluating the factual consistency of abstractive text summarization. *arXiv preprint arXiv:1910.12840* (2019). doi:10.48550/arXiv.1910.12840
- [23] Vivian Lai, Alison Smith-Renner, Ke Zhang, Ruijia Cheng, Wenjuan Zhang, Joel Tetreault, and Alejandro Jaimes. 2022. An exploration of post-editing effectiveness in text summarization. *arXiv preprint arXiv:2206.06383* (2022). doi:10.48550/arXiv.2206.06383
- [24] Junyi Li, Xiaoxue Cheng, Wayne Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. 2023. Halueval: A large-scale hallucination evaluation benchmark for large language models. *arXiv preprint arXiv:2305.11747* (2023). doi:10.48550/arXiv.2305.11747
- [25] Ningke Li, Yuekang Li, Yi Liu, Ling Shi, Kailong Wang, and Haoyu Wang. 2024. HalluVault: A Novel Logic Programming-aided Metamorphic Testing Framework for Detecting Fact-Conflicting Hallucinations in Large Language Models. *arXiv preprint arXiv:2405.00648* (2024). doi:10.48550/arXiv.2405.00648
- [26] Wei Li, Wenhao Wu, Moye Chen, Jiachen Liu, Xinyan Xiao, and Hua Wu. 2022. Faithfulness in natural language generation: A systematic survey of analysis, evaluation and optimization methods. *arXiv preprint arXiv:2203.05227* (2022). doi:10.48550/arXiv.2203.05227
- [27] Ke Liang, Lingyuan Meng, Meng Liu, Yue Liu, Wenxuan Tu, Siwei Wang, Sihang Zhou, Xinwang Liu, Fuchun Sun, and Kunlun He. 2024. A survey of knowledge graph reasoning on graph types: Static, dynamic, and multi-modal. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2024).
- [28] Stephanie Lin, Jacob Hilton, and Owain Evans. 2021. Truthfulqa: Measuring how models mimic human falsehoods. *arXiv preprint arXiv:2109.07958* (2021). doi:10.48550/arXiv.2109.07958
- [29] Zheheng Luo, Qianqian Xie, and Sophia Ananiadou. 2023. Chatgpt as a factual inconsistency evaluator for text summarization. *arXiv preprint arXiv:2303.15621* (2023). doi:10.48550/arXiv.2303.15621
- [30] Potsawee Manakul, Adian Liusie, and Mark JF Gales. 2023. Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. *arXiv preprint arXiv:2303.08896* (2023). doi:10.48550/arXiv.2303.08896
- [31] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2023. Factscore: Fine-grained atomic evaluation of factual precision in long form text generation. *arXiv preprint arXiv:2305.14251* (2023). doi:10.48550/arXiv.2305.14251
- [32] Seungwhan Moon, Pararth Shah, Anuj Kumar, and Rajen Subba. 2019. OpenDialKG: Explainable Conversational Reasoning with Attention-based Walks over Knowledge Graphs. In *Annual Meeting of the Association for Computational Linguistics*. <https://api.semanticscholar.org/CorpusID:196176000>
- [33] Francesco Moramarco, Alex Papadopoulos Korfiatis, Aleksandar Savkov, and Ehud Reiter. 2021. A preliminary study on evaluating consultation notes with post-editing. *arXiv preprint arXiv:2104.04402* (2021). doi:10.48550/arXiv.2104.04402
- [34] R OpenAI et al. 2023. GPT-4 technical report. *ArXiv* 2303 (2023), 08774. doi:10.48550/arXiv.2303.08774
- [35] Shuyin Ouyang, Jie M. Zhang, Mark Harman, and Meng Wang. 2025. An Empirical Study of the Non-Determinism of ChatGPT in Code Generation. *ACM Trans. Softw. Eng. Methodol.* 34, 2, Article 42 (Jan. 2025), 28 pages.
- [36] Libo Qin, Qiguang Chen, Xiachong Feng, Yang Wu, Yongheng Zhang, Yinghui Li, Min Li, Wanxiang Che, and Philip S Yu. 2024. Large language models meet nlp: A survey. *arXiv preprint arXiv:2405.12819* (2024). doi:10.48550/arXiv.2405.12819
- [37] Hongyu Ren and Jure Leskovec. 2020. Beta embeddings for multi-hop logical reasoning in knowledge graphs. *Advances in Neural Information Processing Systems* 33 (2020), 19716–19726.

- [38] Abigail See, Peter J Liu, and Christopher D Manning. 2017. Get to the point: Summarization with pointer-generator networks. *arXiv preprint arXiv:1704.04368* (2017). doi:10.48550/arXiv.1704.04368
- [39] Zeyu Sun, Jie M Zhang, Mark Harman, Mike Papadakis, and Lu Zhang. 2020. Automatic testing and improvement of machine translation. In *Proceedings of the ACM/IEEE 42nd international conference on software engineering*. 974–985.
- [40] Ling Tian, Xue Zhou, Yan-Ping Wu, Wang-Tao Zhou, Jin-Hao Zhang, and Tian-Shu Zhang. 2022. Knowledge graph and knowledge reasoning: A systematic review. *Journal of Electronic Science and Technology* 20, 2 (2022), 100159.
- [41] Neeraj Varshney, Satyam Raj, Venkatesh Mishra, Agneet Chatterjee, Ritika Sarkar, Amir Saeidi, and Chitta Baral. 2024. Investigating and Addressing Hallucinations of LLMs in Tasks Involving Negation. *arXiv preprint arXiv:2406.05494* (2024). doi:10.48550/arXiv.2406.05494
- [42] Neeraj Varshney, Wenlin Yao, Hongming Zhang, Jianshu Chen, and Dong Yu. 2023. A stitch in time saves nine: Detecting and mitigating hallucinations of llms by validating low-confidence generation. *arXiv preprint arXiv:2307.03987* (2023). doi:10.48550/arXiv.2307.03987
- [43] Tu Vu, Mohit Iyyer, Xuezhi Wang, Noah Constant, Jerry Wei, Jason Wei, Chris Tar, Yun-Hsuan Sung, Denny Zhou, Quoc Le, et al. 2023. Freshllms: Refreshing large language models with search engine augmentation. *arXiv preprint arXiv:2310.03214* (2023). doi:10.48550/arXiv.2310.03214
- [44] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. 2023. Can llms express their uncertainty? an empirical evaluation of confidence elicitation in llms. *arXiv preprint arXiv:2306.13063* (2023). doi:10.48550/arXiv.2306.13063
- [45] Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. 2024. Hallucination is inevitable: An innate limitation of large language models. *arXiv preprint arXiv:2401.11817* (2024). doi:10.48550/arXiv.2401.11817
- [46] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600* (2018). doi:10.48550/arXiv.1809.09600
- [47] Jia-Yu Yao, Kun-Peng Ning, Zhen-Hui Liu, Mu-Nan Ning, and Li Yuan. 2023. Llm lies: Hallucinations are not bugs, but features as adversarial examples. *arXiv preprint arXiv:2310.01469* (2023). doi:10.48550/arXiv.2310.01469
- [48] Yifan Yao, Jinhao Duan, Kaidi Xu, Yuanfang Cai, Zhibo Sun, and Yue Zhang. 2024. A survey on large language model (llm) security and privacy: The good, the bad, and the ugly. *High-Confidence Computing* (2024), 100211.
- [49] Chan Xing Yu, Chan Si Yu James, and Poh Hui-Li Phyllis David. [n. d.]. CAN LLMS HAVE A FEVER? INVESTIGATING THE EFFECTS OF TEMPERATURE ON LLM SECURITY. ([n. d.]).
- [50] Jie Zhang, Junjie Chen, Dan Hao, Yingfei Xiong, Bing Xie, Lu Zhang, and Hong Mei. 2014. Search-based inference of polynomial metamorphic relations. In *Proceedings of the 29th ACM/IEEE international conference on Automated software engineering*. 701–712.
- [51] Jie M. Zhang, Mark Harman, Lei Ma, and Yang Liu. 2022. Machine Learning Testing: Survey, Landscapes and Horizons. *IEEE Transactions on Software Engineering* 48, 1 (2022), 1–36.
- [52] Xiaoyu Zhang, Jianping Li, Po-Wei Chi, Senthil Chandrasegaran, and Kwan-Liu Ma. 2023. ConceptEVA: concept-based interactive exploration and customization of document summaries. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–16.
- [53] Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lema Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Yulong Chen, et al. 2023. Siren’s song in the AI ocean: a survey on hallucination in large language models. *arXiv preprint arXiv:2309.01219* (2023). doi:10.48550/arXiv.2309.01219
- [54] Zili Zhou, Shaowu Liu, Guandong Xu, and Wu Zhang. 2019. On completing sparse knowledge base with transitive relation embedding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 3125–3132.

Received 2024-09-13; accepted 2025-01-14